IT457 — CLOUD COMPUTING

Capital One: Cloud Migration and Architecture Case Study

Week 2 Detailed Report

Team member	Student ID
Kavy Sanghani	202301044
Khush Patel	202301142
Rishank Dudhat	202301068

Executive Summary

Capital One's migration to Amazon Web Services (AWS) is a useful case study because the public story connects infrastructure change with software delivery, resilience and engineering practices. Amazon's November 2016 announcement stated that Capital One selected AWS as its predominant cloud infrastructure provider [2]. Capital One's later engineering retrospective describes an earlier cloud decision in 2014 and a final data-center closure in 2021 [3], while AWS's customer story describes the exit from eight on-premises data centers [1]. The differences are reported as separate milestones rather than collapsed into one unambiguous date.

AWS reports that Capital One reduced average development-environment setup from three months to minutes, improved disaster-recovery time in tests by 70%, and reduced both transaction errors and critical-incident resolution time by 50% [1]. These figures are published case-study results, not measurements from this student project. The report reviews the problem, publicly described architecture patterns, service roles, benefits, trade-offs and cost-evidence limitations. It also proposes a controlled educational exercise: a synthetic-data Expense Tracker API first run in Docker locally, then moved to EC2, followed by moving its PostgreSQL database to private Amazon RDS. The lab is planned, not yet deployed or measured.

Scope and Deliverables

This report contains the case background, problem statement, migration analysis, conceptual architecture, outcome and cost analysis, proposed lab architecture, measurement plan, team responsibilities, risks and references. The Capital One diagram is a conceptual synthesis of public sources, not a disclosure of its complete internal production architecture. The prototype will use synthetic data only and will not process actual bank transactions or customer information.

Organization Context, Timeline and Problem Statement

Organization and context

Capital One provides financial services in which digital channels, transaction processing and customer-facing applications are operationally important. In its 2016 announcement, Amazon said Capital One selected AWS as its predominant cloud infrastructure provider and planned to migrate core business and customer applications over the following years [2]. Capital One's own engineering article later describes the decision as beginning in 2014, says testing and development environments were moved to AWS two years later, and dates the final data-center closure to 2021 [3]. AWS's case study describes an eight-data-center exit and an eight-year migration journey [1]. These descriptions refer to different milestones and do not form a perfectly consistent chronology; this report preserves the distinction instead of forcing one date onto every milestone.

Problem statement

Organizations operating traditional data centers must plan physical capacity, procure and maintain hardware, manage facilities, and coordinate infrastructure changes. These responsibilities can slow the creation of development environments and complicate capacity planning when demand changes. The problem is not that on-premises infrastructure is always unsuitable; it is that operational work and fixed capacity can limit the speed with which teams provision resources and deliver software.

For the case study, the central question is: how can a regulated financial institution reduce infrastructure-management friction and improve delivery speed, elasticity and resilience while retaining strong controls over access, data, costs and operations? The public case-study narrative describes Capital One's goal as shifting resources away from infrastructure management toward customer-focused software development [1, 2].

Migration goals identified from the sources

- Provision development and compute capacity faster.
- Support changing workloads without planning every hardware purchase in advance.
- Improve software release frequency and team ownership through agile and DevOps practices.
- Strengthen service resilience and disaster recovery through architecture and testing.
- Use managed and cloud-native services where they fit the application and operating model.
- Build cloud skills and operational readiness across engineering teams.

The case is therefore a combined technology and organizational transformation. Infrastructure services enabled the change, while team skills, delivery practices and operational reviews helped translate those capabilities into repeatable engineering outcomes [1].

Migration Strategy and Technology Approach

From provider selection to cloud-first operations

The 2016 announcement marked the public selection of AWS as Capital One's predominant cloud infrastructure provider [2]. Capital One's 2023 engineering retrospective describes earlier planning in 2014 and the later close of the final data center in 2021 [3]. The practical lesson is that enterprise migration spans years and multiple milestones: selection, landing-zone and skills preparation, application assessments, migration or re-architecture, operational adoption, and data-center exit.

Migration is not one technique

A workload may be moved with limited code changes, partially re-platformed, or substantially re-architected. A lift-and-shift approach may reduce immediate migration effort, but can retain operating burdens such as self-managed databases. Re-platforming can replace selected components with managed services. Re-architecture can change the application boundaries and deployment model, but often requires more design and testing. Public sources show that Capital One's transformation included building many applications in the cloud from the ground up, alongside broader changes to engineering practices [1].

Technology examples in the public record

Service / pattern	Role in the documented examples	Careful interpretation
Amazon EC2	Virtual compute; named in the case study and Eno example [1].	Not every Capital One application runs on EC2.
Amazon S3	Object storage; named among AWS services in the case study [1].	The exact bucket layout and data are not public.
Amazon RDS	Managed relational database service named in the case study [1].	A service example, not proof of a universal bank-wide database design.
AWS Lambda	Event-driven compute in serverless patterns and Eno [1, 3].	Use depends on workload and operational constraints.
DynamoDB	NoSQL data store named in Eno example [1].	Application-specific example.
Route 53, ACM, VPC, load balancer	Parts of a Capital One published API integration pattern [3].	The diagram is a synthesis of one disclosed pattern.
ECS / Fargate, CodeDeploy	Container backends and phased rollout/rollback in the published pattern [3].	Not asserted to be the only runtime or release tool across all apps.

This evidence supports a mixed, workload-specific cloud environment. A sound case analysis should describe the roles of services and the operating principles they enable, while avoiding the implication that every application uses the same stack.

Conceptual View of Capital One's Published Architecture Patterns

Figure 1 synthesizes a request path described in Capital One's engineering article: Route 53 provides DNS and regional routing; a public API endpoint uses TLS; a private load balancer and VPC-based private integration connect to backend services such as Lambda or Fargate. The article also discusses phased releases and rollback using CodeDeploy [3]. AWS's customer case study separately names EC2, S3 and RDS among the services used or planned as part of the broader transformation, and identifies EC2, Lambda and DynamoDB in the Eno example [1].

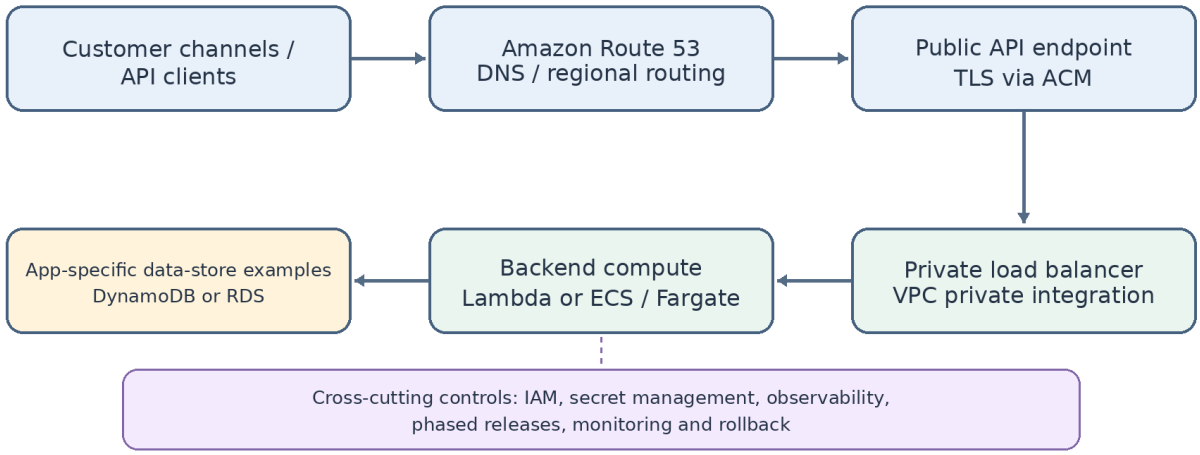


Figure 1. Conceptual synthesis of public AWS and Capital One descriptions [1, 3]. This is not Capital One's complete internal production architecture.

How to read the diagram

- The client/API path illustrates a published API integration pattern; Route 53 and TLS support routing and protected communication.
- The load balancer and VPC-based integration show a network boundary between the public entry point and private application backends.
- Lambda and Fargate are alternative backend compute patterns in the described API architecture; they do not have to be used together for every request.
- DynamoDB and RDS are shown as application-dependent data-store examples drawn from separate public examples, not as a universal paired database layer.
- IAM, secrets management, observability and phased deployment are supporting controls that help teams operate applications safely.

The diagram is deliberately labelled conceptual. Public case studies do not disclose every internal subnet, route, security policy, data flow, service dependency or regulatory control. A precise replica would therefore be speculative and is outside the scope of this project.

Architecture Analysis: Security, Resilience and Operations

Resilience and availability

Capital One's AWS case study says the company adopted an active-active architecture with automated failover for its resilience approach and reported a 70% reduction in disaster-recovery time in tests [1]. This should be understood as a reported outcome of a broader programme rather than a guarantee that every service has zero downtime. Active-active designs require careful data consistency, traffic routing, failure detection, and operational exercises.

Security and shared responsibility

AWS describes cloud security as a shared responsibility. AWS manages security of the underlying cloud infrastructure, while customer duties vary with the service and include responsibilities such as guest operating system management, application software and configuring security controls for services such as EC2 [5]. For a banking environment, identity, least privilege, data classification, encryption, secret management, logging, change control and recovery testing are core design concerns. This report does not claim that public articles expose Capital One's full security-control implementation.

Private networking for the student lab

The proposed lab places the API on an EC2 instance with narrowly scoped ingress and places the RDS database in private subnets. AWS documentation says private RDS instances are reachable from inside the VPC and have no public IP, reducing exposure to the public internet [8]. Database inbound rules should permit only the application's security group, while the instance's administration path should be restricted to a trusted address or managed access method [11]. Use individual access identities, minimum permissions and secure secret handling [9].

Operational excellence and observability

The AWS Well-Architected Framework describes six pillars: operational excellence, security, reliability, performance efficiency, cost optimization and sustainability [4]. For this project, the practical equivalents are repeatable deployment steps, a documented rollback or restart procedure, measured response times and failures, a cost estimate, and a resource cleanup checklist. CloudWatch can collect resource and application telemetry when it is configured [10].

Trade-off

Managed services reduce some operational work but do not remove the need for secure configuration, monitoring, capacity decisions and cost control. The most appropriate architecture is the one that meets workload needs with understood risks and an acceptable operational and cost profile—not the one with the most services.

Reported Results, Cost Considerations and Evidence Limits

Outcomes reported in the AWS customer case study

Reported item	Published result	How this report interprets it
Data-center exit	Eight on-premises data centers closed [1].	Historical outcome reported by AWS.
Development environment setup	Average build time changed from three months to minutes [1].	Provisioning/process improvement; not a lab result.
Disaster recovery	70% lower recovery time in tests [1].	Reported test outcome; not a guarantee of zero downtime.
Transaction errors	50% reduction [1].	Reported case-study metric.
Critical incident resolution	50% reduction [1].	Reported case-study metric.
Release cadence	From monthly/quarterly updates to multiple code releases per day [1].	Describes delivery practices and technical enablement together.

Cost: what can and cannot be concluded

The public sources reviewed do not provide a verified total dollar figure for the entire Capital One migration. For this reason, the report avoids stating a made-up migration budget or asserting that the entire enterprise saved a specific percentage. EC2 On-Demand charges depend on configured instance and time of use, while other billable items may include storage, public IPv4, data transfer, monitoring and database services [6]. AWS Pricing Calculator is the appropriate starting point for a workload-specific estimate [7].

The team's lab cost model should compare the configurations using one AWS Region and a clearly stated run duration. Scenario A includes EC2, attached storage, and the containerized database; Scenario B includes EC2 plus RDS, storage/backups and related monitoring/traffic. The managed-service scenario may have a higher direct service bill but lower database administration effort. The comparison must report both cost and operational effort rather than assuming RDS is automatically cheaper.

Evidence limitations

- The principal outcome metrics are published in an AWS customer case study and are not independently measured by our group [1].
- The 2016 press announcement describes selection of AWS as the predominant provider [2], while a 2023 Capital One article places the cloud decision in 2014 and the final data-center closure in 2021 [3]. These are different milestones and the public timeline should be reported with that distinction.
- The public architecture article describes selected patterns, not the entire private production architecture [3].
- A small synthetic-data lab cannot reproduce banking-scale traffic, regulatory context, internal platforms or Capital One's organization-wide migration.

The evidence supports a cautious conclusion: the case study associates the cloud transformation with faster provisioning, delivery and improved resilience, but cost, security and reliability outcomes depend on architecture and operating practices.

Proposed Practical Exercise: Local Docker to AWS

Exercise objective

The planned practical exercise moves a simple application from a local Docker container to a cloud VM, then refactors part of it to use a managed service and documents effort and cost differences. Our selected prototype is a small Expense Tracker API that uses synthetic records only; it is not a banking product and will not connect to any real account or customer data.

Application scope

A Node.js/Express REST API will expose basic endpoints such as `GET /health`, `GET /expenses`, `POST /expenses` and `DELETE /expenses/:id`. PostgreSQL stores a small deterministic test dataset. The same application build, schema, request mix and load-test script will be used across each test phase to keep comparisons meaningful.

Three deployment states

State	Architecture	Question to answer
Baseline	Local Docker Compose: API container + PostgreSQL container.	What is the response-time/error baseline on the development machine?
Phase A: Lift-and-shift	Same API and PostgreSQL containers on one EC2 instance; persistent storage for database files.	What changes when the same containerized stack runs on a cloud VM?
Phase B: Managed database	API container remains on EC2; PostgreSQL moves to Amazon RDS with private connectivity.	How does a managed database change operational effort, reliability controls and cost?

Why this prototype

A compact REST API is realistic enough to demonstrate deployment, connectivity and load testing but small enough to debug and repeat. Separating the API and database lets the group discuss compute, persistence, network security and managed services without building an unnecessarily complex application.

Prototype boundaries

- Use dummy expense rows with no names, account numbers, cards or personal financial data.
- Do not integrate payment gateways, real bank APIs or third-party account access.
- Keep an RDS database private and restrict its security group to the application instance.
- Avoid optional infrastructure such as a load balancer or multi-Region setup until the basic experiment works and the budget is understood.
- Document every deviation from the planned architecture and test protocol.

Proposed Lab Architecture

Figure 2 describes the proposed educational application architecture for the RDS-refactored state. The API runs in Docker on a small EC2 instance in a VPC. A browser/API client and k6 load generator send controlled requests to the application. The application connects to PostgreSQL on RDS over a private network path; the database should not be publicly accessible. Logs and resource metrics can be collected using CloudWatch and host/container tools where configured [8, 10].

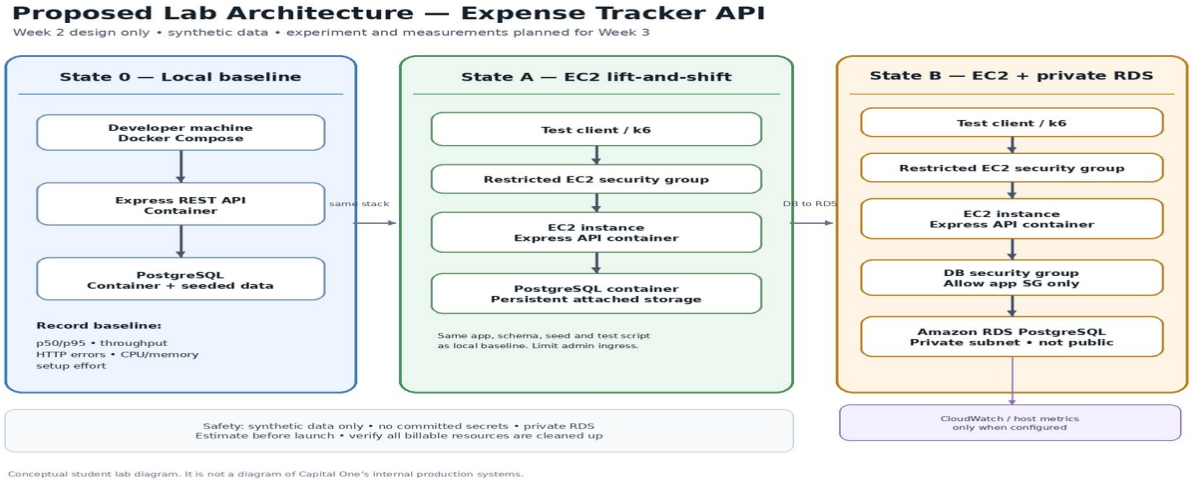


Figure 2. Proposed lab architecture; it is a design target, not a deployment already completed.

Networking and access assumptions

- The EC2 instance accepts only the necessary application traffic. SSH/administrative access is restricted to a trusted source or uses a managed access method; SSH is never opened to the whole internet.
- RDS uses a DB subnet group with private subnets spanning Availability Zones, while the prototype can use a Single-AZ DB deployment if selected for budget and availability requirements.
- The RDS security group allows the PostgreSQL port only from the EC2 application security group, not from arbitrary internet addresses [8, 11].
- Database credentials are supplied at runtime through an ignored local environment file or a secrets-management service; secret values are never committed to GitHub.
- CloudWatch logging/metrics should be configured deliberately because monitoring and log storage can add cost [10].

Migration steps

- Containerize the API and database locally and verify the seeded dataset.
- Record a baseline test run and exact app/config versions.
- Launch a small EC2 instance, deploy the same containers and verify the API.
- Provision RDS PostgreSQL with private access; migrate the schema and synthetic data, update the connection configuration and verify connectivity.
- Run the same checks and load-test script after each change.
- Review AWS billing/estimates, then stop or delete billable resources after collecting results.

This lab architecture intentionally omits multi-Region routing, auto scaling and an Application Load Balancer. Those components are relevant to enterprise patterns but would add configuration and cost before the core migration experiment has been validated.

Experimental Methodology and Evaluation Plan

Reproducibility

Use the same API version, route mix, data seed, k6 script and load profiles for the local baseline, EC2-only phase and EC2-plus-RDS phase. Record region, instance type, database configuration, container image tag, database settings, test date, network location, warm-up time and run duration. Repeat each profile three times where feasible, then report the median and spread. If a run is interrupted or a configuration changes, record that fact rather than silently mixing results.

Proposed load profiles

Profile	Suggested load	Duration / notes
P1	1 virtual user	Warm-up plus 2-minute steady request period. Establish functional and low-load behaviour.
P2	5 virtual users	Same request mix and dataset; observe concurrency impact.
P3	10 virtual users	Run only if the small instance remains stable; reduce load if error rates or resource saturation become excessive.

Metrics to collect

- Latency: p50 and p95 response time in milliseconds.
- Throughput: successful requests per second, with virtual-user profile stated.
- Error rate: failed or unexpected HTTP responses divided by total requests.
- Resource usage: CPU and memory from host/container tools and CloudWatch when configured.
- Database behaviour: API-observed request/query time and connection errors; describe the measurement method.
- Deployment effort: elapsed setup time, number of manual steps and interventions.
- Cost: calculator estimate and actual resource uptime; show short lab-period cost separately from a 730-hour monthly equivalent [6, 7].
- Recovery check: restart the API container and measure time until the health endpoint succeeds again. This is not a production disaster-recovery test.

Analysis approach

Present a comparison table for the three states with all metrics and configuration notes. Keep latency units consistent and describe outliers. Identify confounding factors such as network distance, instance size, background activity and differences between local and cloud hardware. Directly compare like-for-like runs and do not state that managed RDS must be faster or cheaper: the purpose is to measure trade-offs between a self-managed database container and a managed database service.

Proposed completion checks

- Expected functional tests pass for the synthetic dataset.
- Each configuration has recorded p50/p95, throughput and error rate.
- Three repeat runs are completed per load profile where feasible, or deviations are explained.
- Cost inputs, Region, uptime and calculation assumptions are saved in the repository.
- RDS public access is disabled and database ingress is narrowly scoped [8, 9, 11].
- All billable resources are checked after experiments for cleanup.

These are future checks. No test measurements or deployment outcomes are claimed in Week 2.

Week 2 Contributions, Week 3 Plan, Risks and References

Week 2 contributions (completed and merged)

Week 2 contributor	Work completed and merged	Key deliverables
Kavy Sanghani (202301044)	Case background, problem/goals, published outcomes and costs; source index; architecture/service analysis and conceptual diagram; report integration.	01-case-background.md; 02-problem-and-migration-goals.md; 03-architecture-and-aws-services.md; 04-outcomes-costs-and-tradeoffs.md; sources.md; conceptual architecture files; final report.
Rishank Dudhat (202301068)	Practical exercise, measurement protocol, risks/limitations, lab architecture and Week 2 handoffs.	05-practical-exercise-plan.md; 06-measurement-risk-and-limitations.md; lab-architecture files; team-roles.md.

Planned Week 3 responsibilities

These roles are forward-looking and do not imply that the deployment or measurements are already complete.

- Kavy Sanghani (202301044): Coordinate the lab architecture and access/security choices, help validate deployment states, and integrate verified results into the report.
- Khush Patel (202301142): Support API functional checks and repeatable test runs, help record observations, and assist with the cost comparison and results interpretation.
- Rishank Dudhat (202301068): Lead the Docker-to-EC2-to-private-RDS implementation, maintain the load-test protocol, record measurements and deviations, and verify AWS resource cleanup.

Main risks and responses

- Unexpected AWS charges: estimate before launching, keep resources small, use budget alerts where available, record uptime, and clean up after testing.
- Credential leakage: use least-privilege access and runtime secrets; never commit secret keys or `.env` values [5, 9].
- Public database exposure: private RDS access, narrow security-group rules and secure authentication [8, 11].
- Weak comparisons: repeat runs and preserve test settings; label any deviations.
- Overclaiming: distinguish Capital One's reported results from our prototype observations.
- Scope growth: first complete the single-VM and RDS comparison before attempting load balancing or multi-Region features.

Conclusion

Capital One's public case study shows that cloud migration can be a broad organizational and architectural transformation. The reported gains in provisioning, release frequency and resilience are associated with cloud services as well as changes in engineering skills, delivery practices and operational readiness [1]. For our project, the next step is to turn that case analysis into a controlled, small-scale experiment. The proposed Docker-to-EC2-to-RDS sequence will let us evaluate the difference between moving a workload and modernizing a component, with evidence drawn from our own measurements rather than assumed outcomes.

References

[1] AWS, 'Capital One Completes Migration from Data Centers to AWS'. https://aws.amazon.com/solutions/case-studies/capital-one-all-in-on-aws/

[2] Amazon Press Center, 'Capital One Selects AWS as its Predominant Cloud Infrastructure Provider' (28 November 2016). https://press.aboutamazon.com/2016/11/capital-one-selects-aws-as-its-predominant-cloud-infrastructure-provider

[3] Capital One Tech, 'The serverless-first strategy experience'. https://www.capitalone.com/tech/cloud/serverless-first-strategy/

[4] AWS, 'The pillars of the AWS Well-Architected Framework'. https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html

[5] AWS, 'Shared Responsibility Model'. https://aws.amazon.com/compliance/shared-responsibility-model/

[6] AWS, 'Amazon EC2 On-Demand Pricing'. https://aws.amazon.com/ec2/pricing/on-demand/

[7] AWS, 'AWS Pricing Calculator Documentation'. https://docs.aws.amazon.com/pricing-calculator/

[8] AWS, 'Setting up public or private access in Amazon RDS'. https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/security-public-private.html

[9] AWS, 'Security best practices for Amazon RDS'. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_BestPractices.Security.html

[10] AWS, 'What is Amazon CloudWatch?'. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/

[11] AWS, 'Amazon EC2 security groups'. https://docs.aws.amazon.com/us_en/AWSEC2/latest/UserGuide/ec2-security-groups.html